

```
#include <FS.h>
#include <Wire.h>
#include <ArduinoJson.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_SH110X.h>
#include <Adafruit_seesaw.h>
#include <DFRobot_GR10_30.h>
#include <DFRobot_MAX17043.h>
#include <DFRobot_VisualRotaryEncoder.h>
#include <DFRobot_BloodOxygen_S.h>
#include <SparkFun_Qwiic_Keypad_Arduino_Library.h>
#include <PN532_I2C.h>
#include <NfcAdapter.h>
#include <Adafruit_EEPROM_I2C.h>
```

```
const String version = "8";
const int devicesCount = 8;
const int programsCount = 8;
```

```
String action = "";
int transitionType = 0;
int channelsCount;
int surfacesCount;
int activeProgram = -1;
int lastProgramIndex = -1;
```

```
#include "../engine/Storage.h";
Storage storage = Storage();
```

```
#include "../engine/I2C.h";
I2C i2c = I2C();
```

```
#include "../engine/Program.h";
Program * programs[programsCount];
```

```
#include "../engine/Device.h";
Device * devices[devicesCount];
int deviceIndexGamepad = -1;
int deviceIndexAccelerometer = -1;
int deviceIndexNfc = -1;
int deviceIndexKeypad = -1;
int deviceIndexMemory = -1;
int lastDeviceIndex = -1;
```

```
#include "../devices/AM.h";
#include "../devices/Gamepad.h";
#include "../devices/Keypad.h";
#include "../devices/NFC.h";
#include "../devices/DistanceSensor.h";
#include "../devices/Memory.h";
```

```
#include "../engine/Pixel.h";
#include "../engine/OLED.h";
#include "../engine/Port.h";
#include "../engine/Channel.h";
Channel * channels;
```

```
#include "../engine/Surface.h";
Surface * surfaces;
```

```
#include "../engine/Transition.h";
Transition transition = Transition();
```

```
#include "../programs/Batterfly.h";
#include "../programs/Gravity.h";
#include "../programs/VV.h";
#include "../programs/Logo.h";
#include "../programs/Kontrolery.h";
#include "../programs/Skaner3d.h";
#include "../programs/Battery.h";
#include "../programs/Contacts.h";
```

```

#include "./engine/Menu.h";
Menu menu;

void setup() {
    Serial.begin(9600);
    storage.init();
    i2c.init();

    const String path = "/config/surfaces/" + version + ".json";
    JsonArray & config = storage.load(path);
    surfacesCount = config.size();

    channelsCount = Channel::count(config);
    channels = new Channel[channelsCount];
    for (int i = 0; i < channelsCount; i++)
        channels[i].init(i);

    Kontrolery * kontrolery = new Kontrolery();
    kontrolery->skanuj();

    for (int i = 0; i <= lastDeviceIndex; i++)
        devices[i]->init();

    surfaces = new Surface[surfacesCount];
    for (int i = 0; i < surfacesCount; i++) {
        Surface * surface = new Surface();
        surface->init(config[i]);
        surfaces[i] = * surface;
    }

    menu.init();
}

void loop() {
    for (int i = 0; i <= lastDeviceIndex; i++)
        devices[i]->tick();

    for (int i = 0; i < channelsCount; i++)
        channels[i].tick();

    for (int i = 0; i < surfacesCount; i++)
        surfaces[i].tick();

    menu.tick();
    transition.tick();

    for (int i = 0; i <= lastProgramIndex; i++)
        programs[i]->tick();

    for (int i = 0; i < surfacesCount; i++)
        surfaces[i].clear();

    for (int i = 0; i <= lastProgramIndex; i++)
        programs[i]->draw();

    menu.draw();
    transition.draw();

    for (int i = 0; i < surfacesCount; i++)
        surfaces[i].draw();
}

```